

Matplotlib

June 17, 2020

1 Matplotlib

[Matplotlib](#) is a plotting library for python. It is a very powerful plotting library useful with Python and NumPy. It was also used in while taking the image of blackhole. The most used module of Matplotlib is Pyplot which provides an interface like MATLAB but instead.

Below is the description given by the [NumFocus](#) group which maintains the projects that is widely used in data analysis like pandas, numpy, SciPy, matplotlib and so on. Matplotlib is a Python 2D plotting library which produces publication-quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shell, the Jupyter notebook, web application servers, and seven graphical user interface toolkits. For simple plotting, the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. Full control of line styles, font properties, axes properties, etc. is accessible via an object-oriented interface.[NumFocus](#)

1.1 Installing

Details description on the installation steps can be found [here](#). It can be simply installed using the python package manager called `pip` using the command below

```
python -m pip install -U pip
```

When the matplotlib is imported, generally it is given an alias `plt`. It can be imported as

```
[1]: from matplotlib import pyplot as plt
```

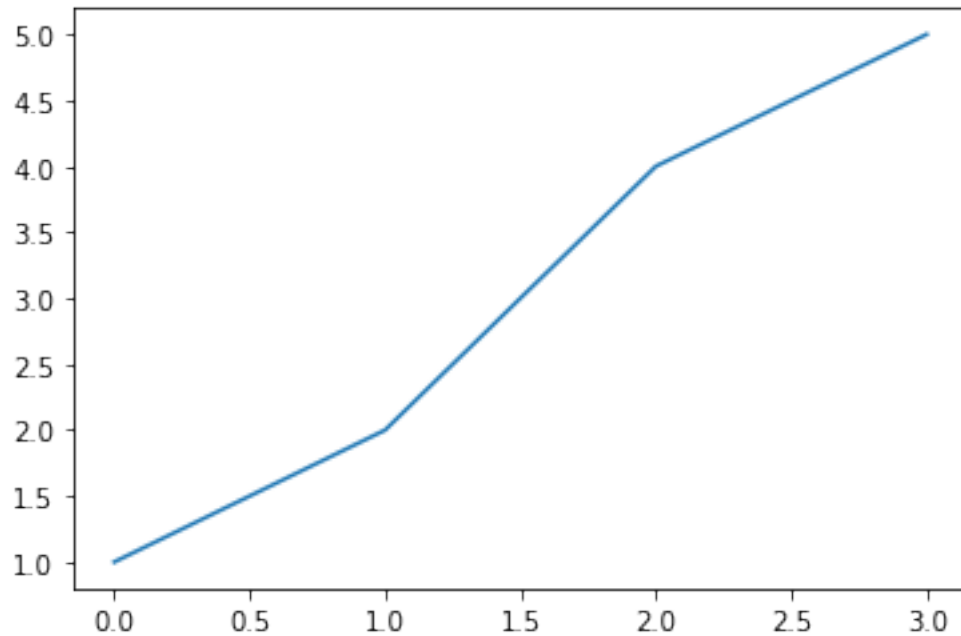
OR

```
[2]: import matplotlib.pyplot as plt
```

Lets create a simple line plot

```
[3]: plt.plot([1,2,4,5])
```

```
[3]: [<matplotlib.lines.Line2D at 0x1c947808f60>]
```



If we are working outside jupyter notebook environment then the above code block will not create any plot. We need to issue another statement saying we want to print the plot.

Lets see what do we mean by this with an example. A file `plot.py` is created with follwing lines of code

```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4,5])
```

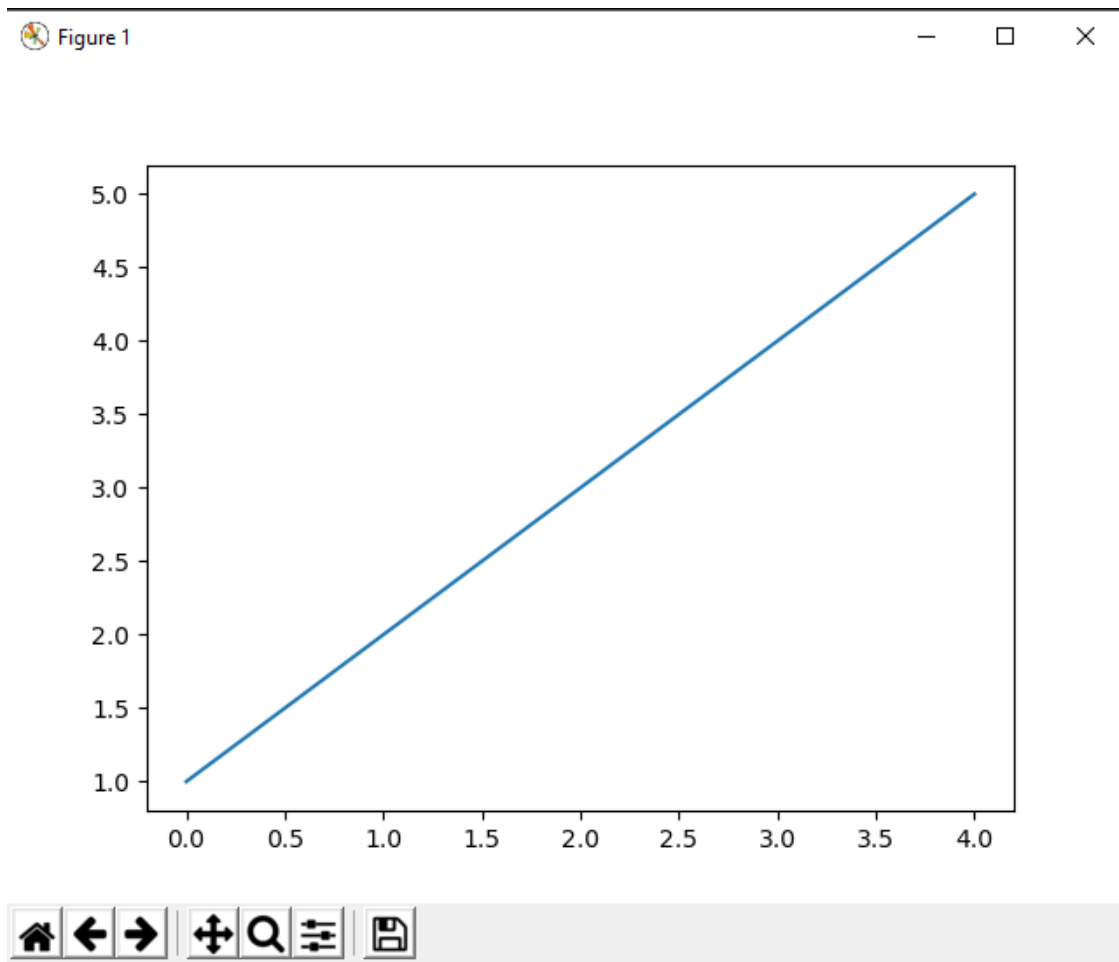
If we run the python script in the file then we will get nothing. The script will be executed and cursor moves to next line without showing any

```
> python plot.py
>
```

If we modify the script as

```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4,5])
plt.show()
```

then we will get following plot as result of execcuting the python script in the `plot.py` file.



We can customize how the plot looks using `style` package. It allows us to select the style of plot. Lets see what are the available styles.

```
[4]: print(plt.style.available)

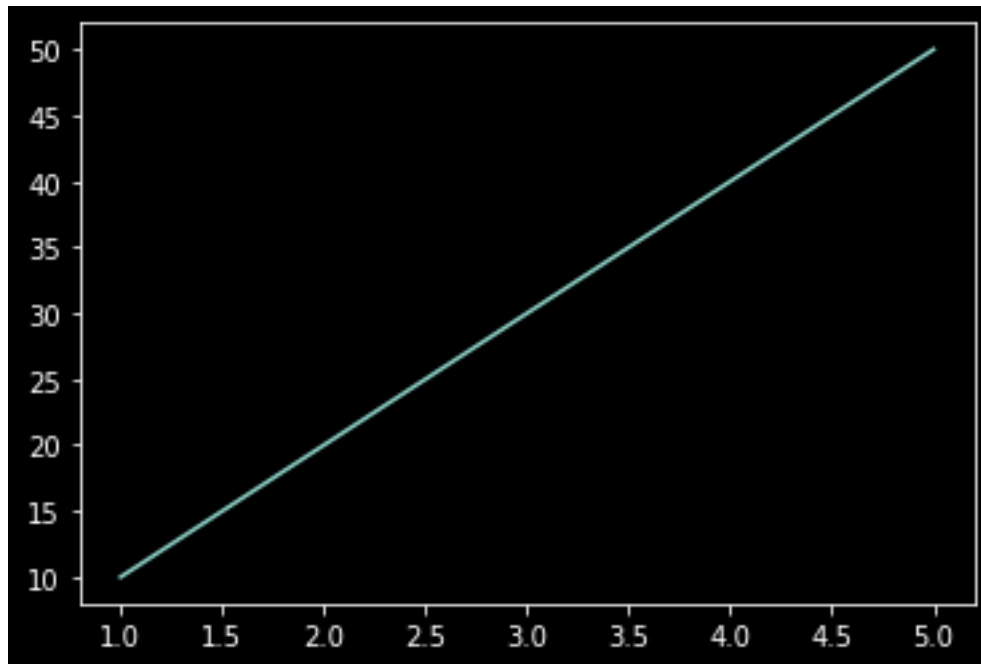
['bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot',
'grayscale', 'seaborn-bright', 'seaborn-colorblind', 'seaborn-dark-palette',
'seaborn-dark', 'seaborn-darkgrid', 'seaborn-deep', 'seaborn-muted', 'seaborn-
notebook', 'seaborn-paper', 'seaborn-pastel', 'seaborn-poster', 'seaborn-talk',
'seaborn-ticks', 'seaborn-white', 'seaborn-whitegrid', 'seaborn',
'Solarize_Light2', 'tableau-colorblind10', '_classic_test']
```

We will try some of them with different types of plot

```
[5]: import matplotlib.pyplot as plt
plt.style.use('dark_background')
```

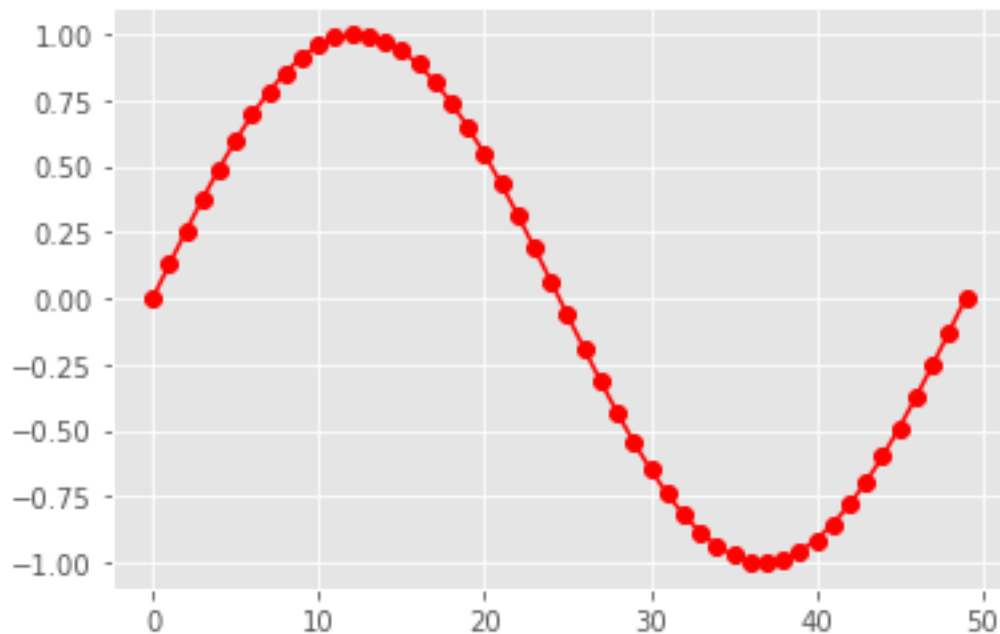
```
[6]: plt.plot([1,2,3,4,5],[10,20,30,40,50])
```

```
[6]: [<matplotlib.lines.Line2D at 0x1c9529297f0>]
```



We can also specify style for specific plot only. We will use ggplot style. Ggplot is a popular plotting library in programming language called R.

```
[7]: import numpy as np
with plt.style.context('ggplot'):
    plt.plot(np.sin(np.linspace(0, 2 * np.pi)), 'r-o')
plt.show()
```



We can add title, labels in X and Y axis legends which are useful for better describing the plot. Let see following example.

```
[8]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
plt.style.use('seaborn')
```

Loading the dataset that will be used by plotting

```
[9]: df = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/
↳solar.csv", error_bad_lines=False)
```

```
[10]: df.head()
```

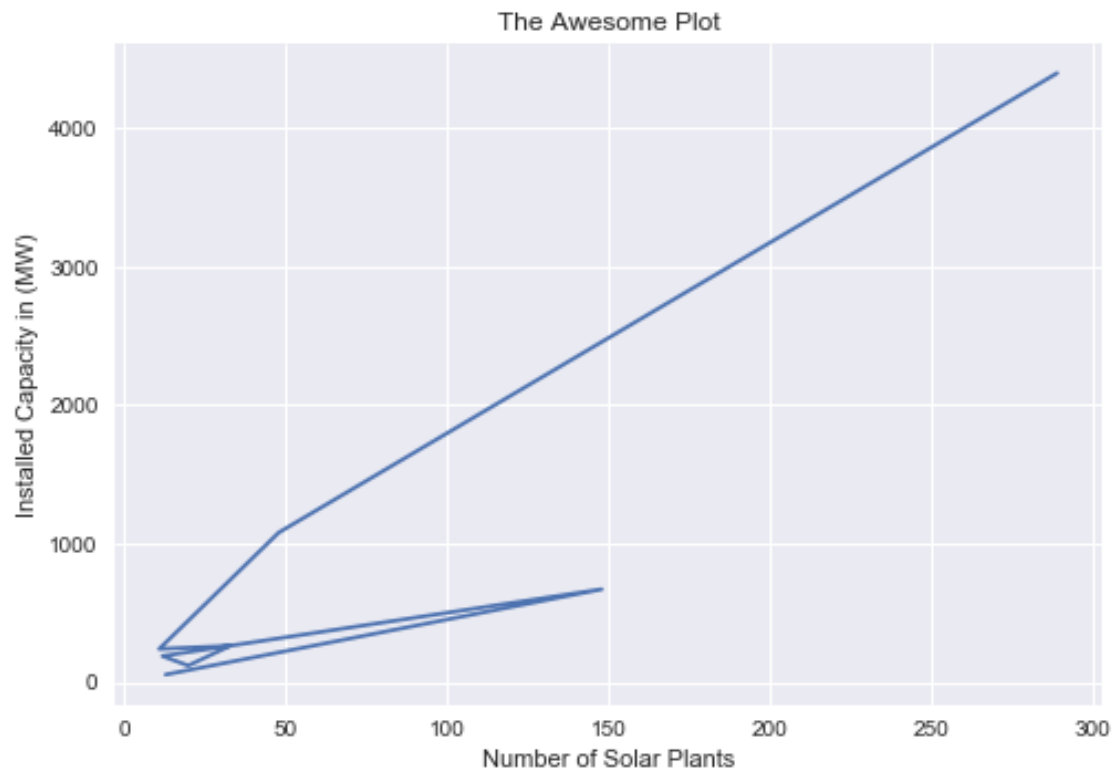
```
[10]:
```

	State	Number of Solar Plants	Installed Capacity (MW)	\
0	California	289	4395	
1	Arizona	48	1078	
2	Nevada	11	238	
3	New Mexico	33	261	
4	Colorado	20	118	

	Average MW Per Plant	Generation (GWh)
0	15.3	10826
1	22.5	2550
2	21.6	557
3	7.9	590
4	5.9	235

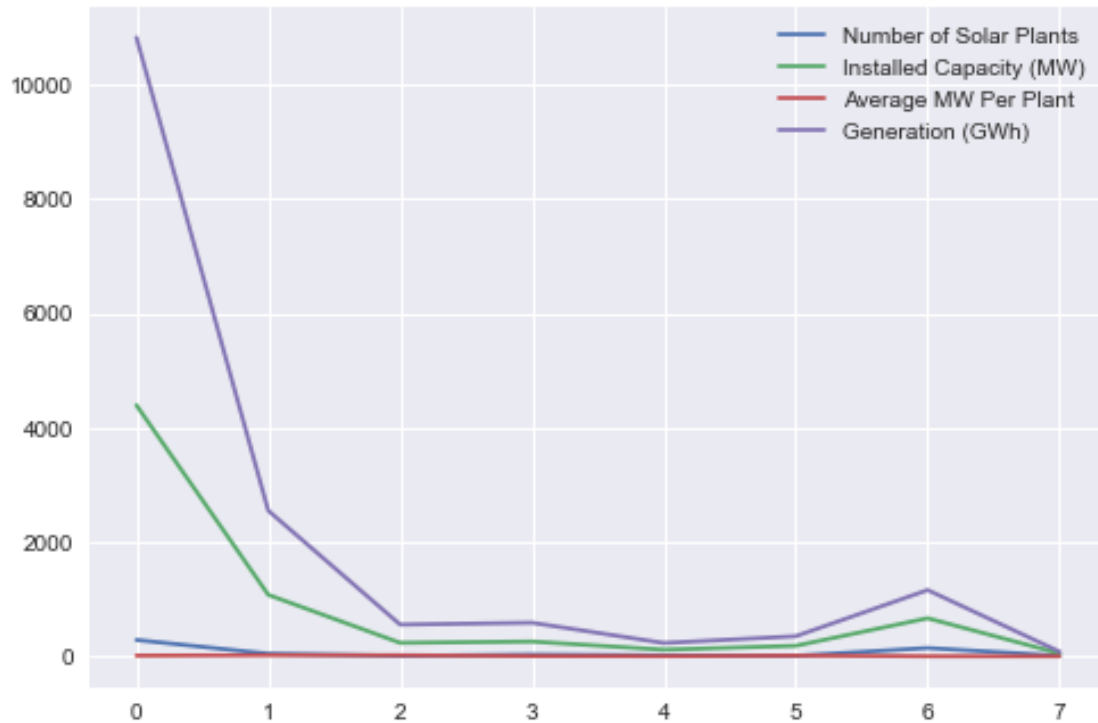
Creating a plot with labels in X-axis, Y-axis and title.

```
[11]: plt.plot(df['Number of Solar Plants'], df['Installed Capacity (MW)'])
plt.xlabel('Number of Solar Plants')
plt.ylabel('Installed Capacity in (MW)')
plt.title('The Awesome Plot')
plt.show()
```



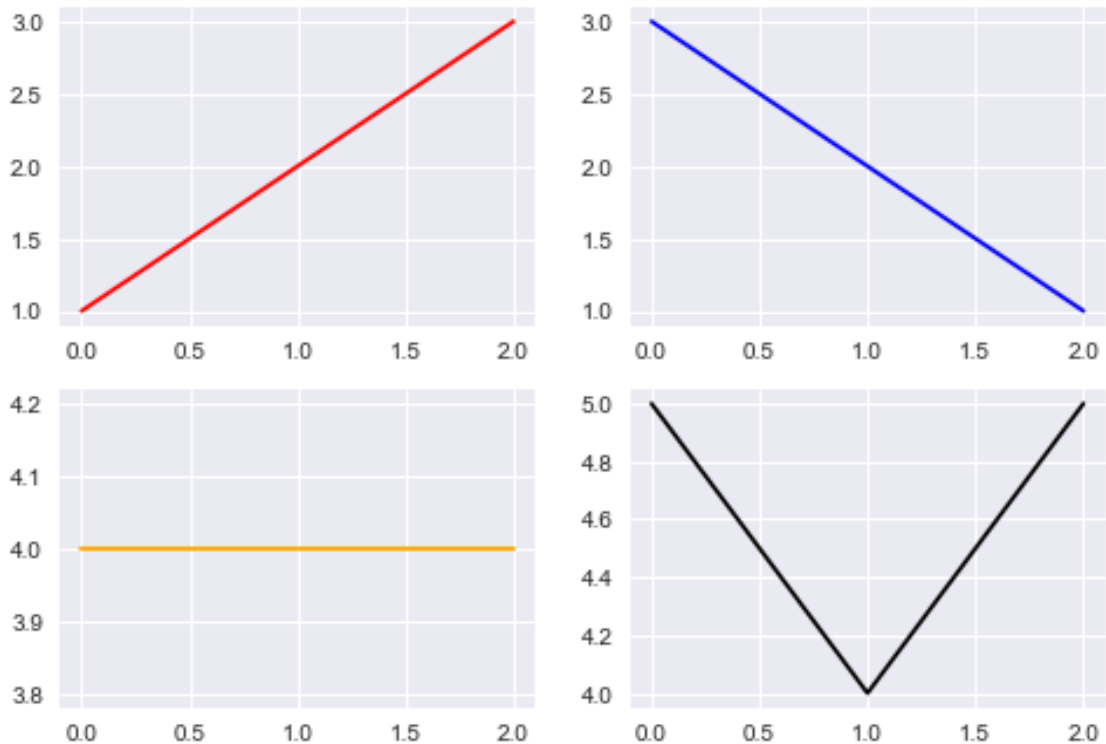
We can also directly plot from dataframe

```
[12]: df.plot()  
plt.legend()  
plt.show()
```



We can also plot multiple plot in a single figure as follows

```
[13]: fig = plt.figure()
      # create a 4 plots and use tuple unpacking to name everyplot
      (ax1, ax2), (ax3, ax4) = fig.subplots(2,2)
      ax1.plot([1,2,3], color = "red")
      ax2.plot([3,2,1], color = "blue")
      ax3.plot([4,4,4], color = "orange")
      ax4.plot([5,4,5], color = "black")
      plt.show()
```



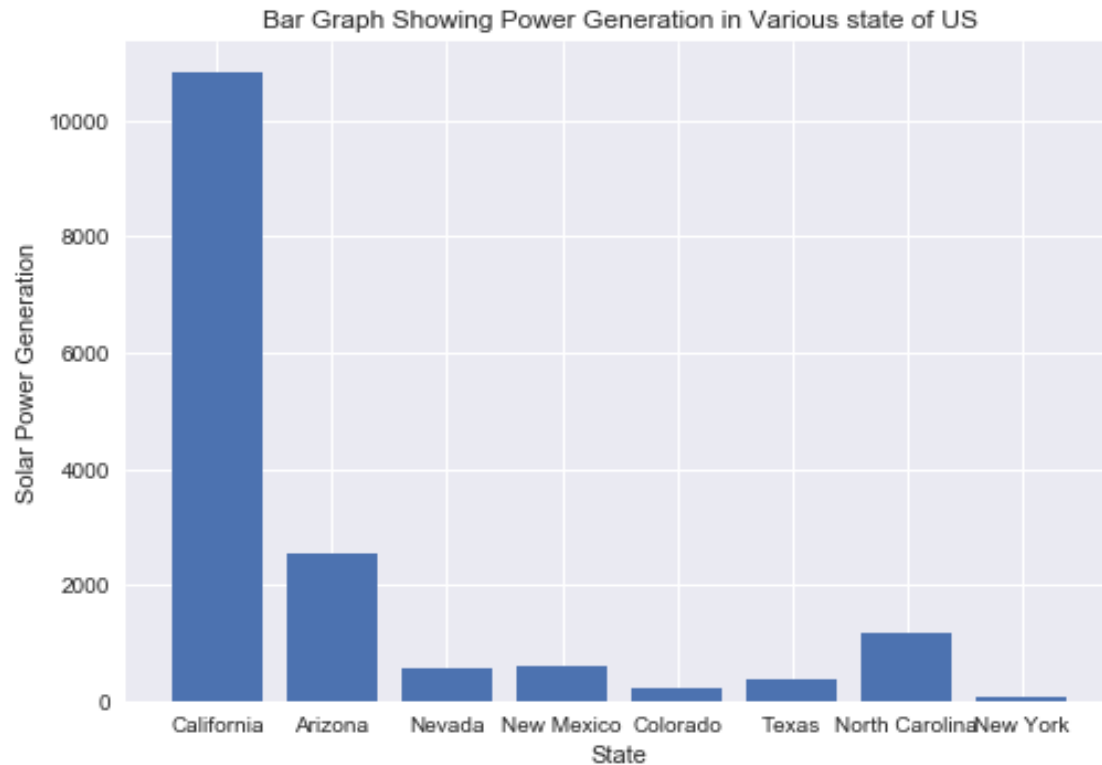
Now we will see how we can plot various kinds of graph using matplotlib.

1.2 Bar Graph

A bar graph is a graph that shows categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally.

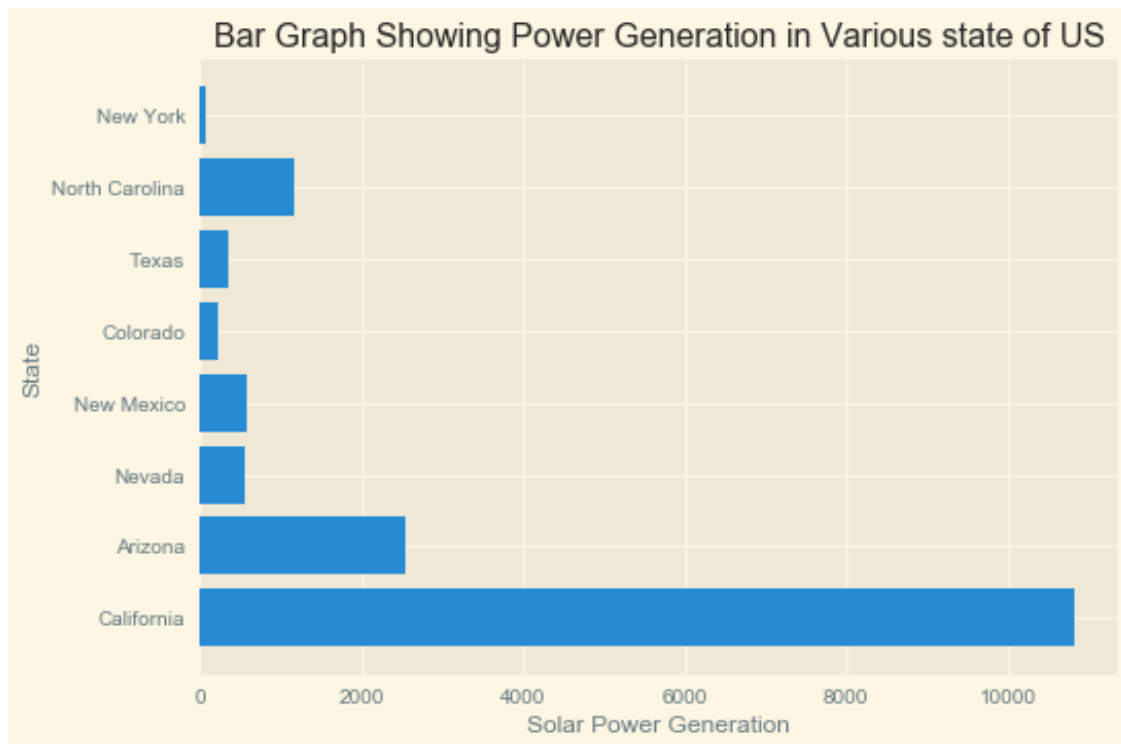
```
[14]: df=df.set_index('State')
```

```
[15]: plt.bar(df.index,df['Generation (GWh)'])
plt.ylabel('Solar Power Generation')
plt.xlabel('State')
plt.title('Bar Graph Showing Power Generation in Various state of US')
plt.show()
```



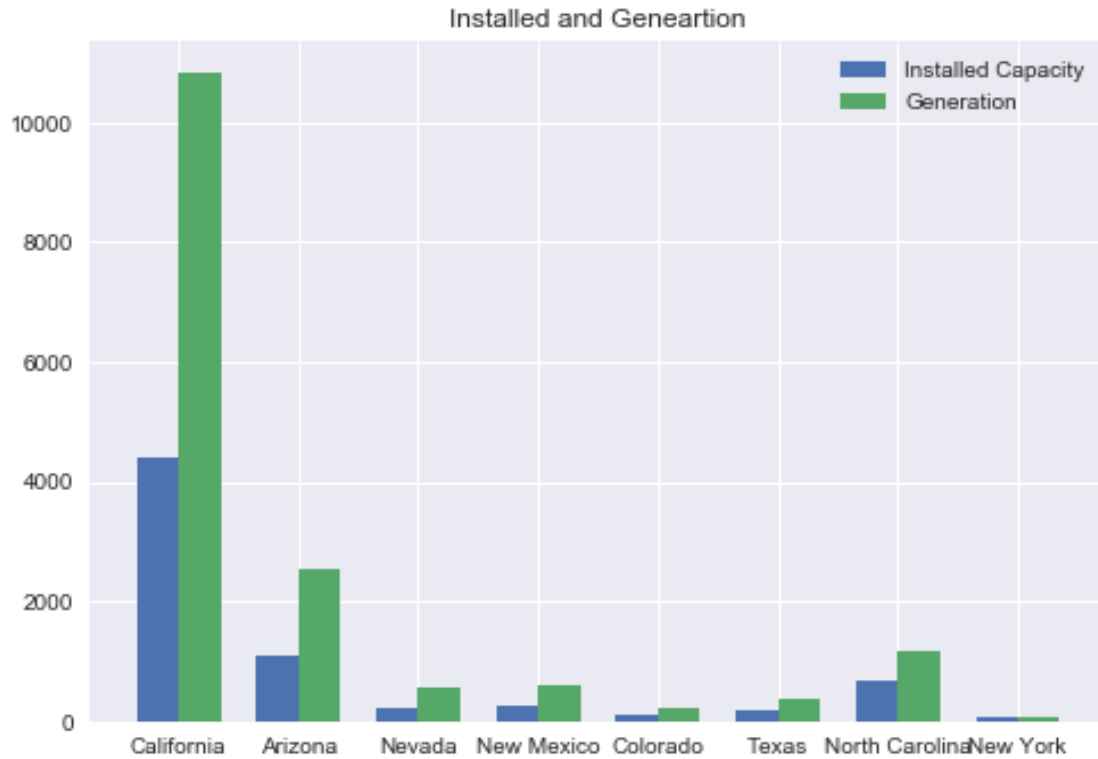
The example above is of vertical bar graph. We can use `barh` method for horizontal bar graph. We will be also showcasing how we can set the temporary style for a particular graph.

```
[16]: with plt.style.context('Solarize_Light2'):
      plt.barh(df.index,df['Generation (GWh)'])
      plt.xlabel('Solar Power Generation')
      plt.ylabel('State')
      plt.title('Bar Graph Showing Power Generation in Various state of US')
      plt.show()
```



We can plot multiple bar graph for same category. These type of bar graph is called grounded bar graph

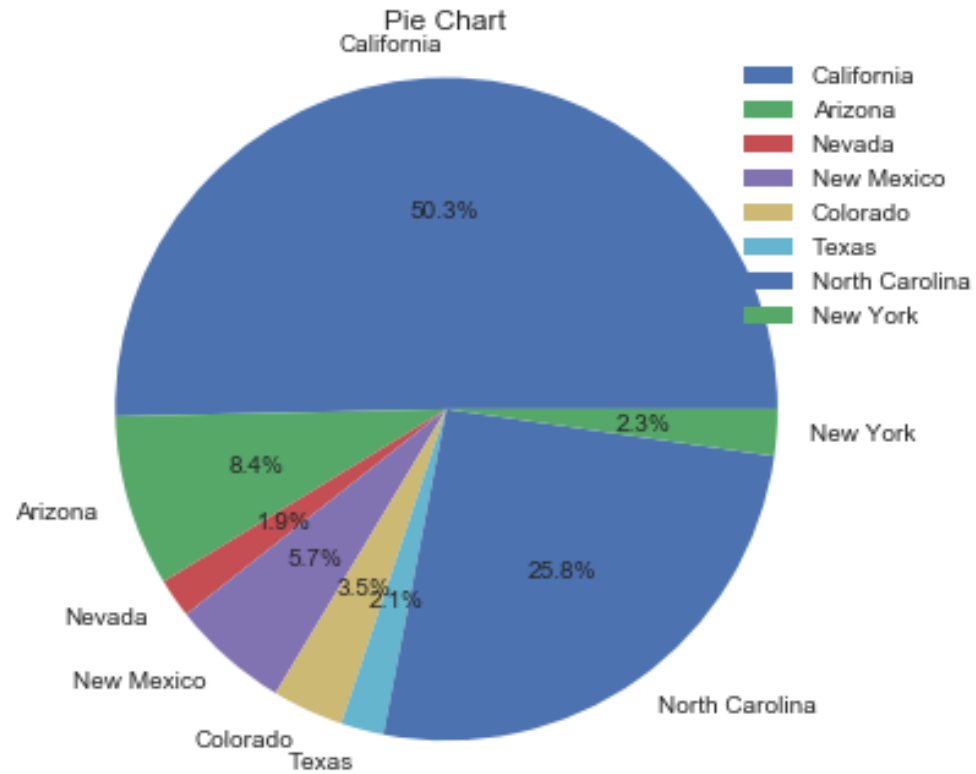
```
[17]: labels = df.index
x = np.arange(len(labels)) # the label locations
width = 0.35 # the width of the bars
fig, ax = plt.subplots()
rects1 = ax.bar(x - width/2, df['Installed Capacity (MW)'], width, label='Installed Capacity')
rects2 = ax.bar(x + width/2, df['Generation (GWh)'], width, label='Generation')
ax.set_title('Installed and Generation')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
plt.show()
```



1.2.1 Pie Chart

A pie chart displays data, information, and statistics in an easy-to-read ‘pie-slice’ format with varying slice sizes telling you how much of one data element exists. The bigger the slice, the more of that particular data was gathered.

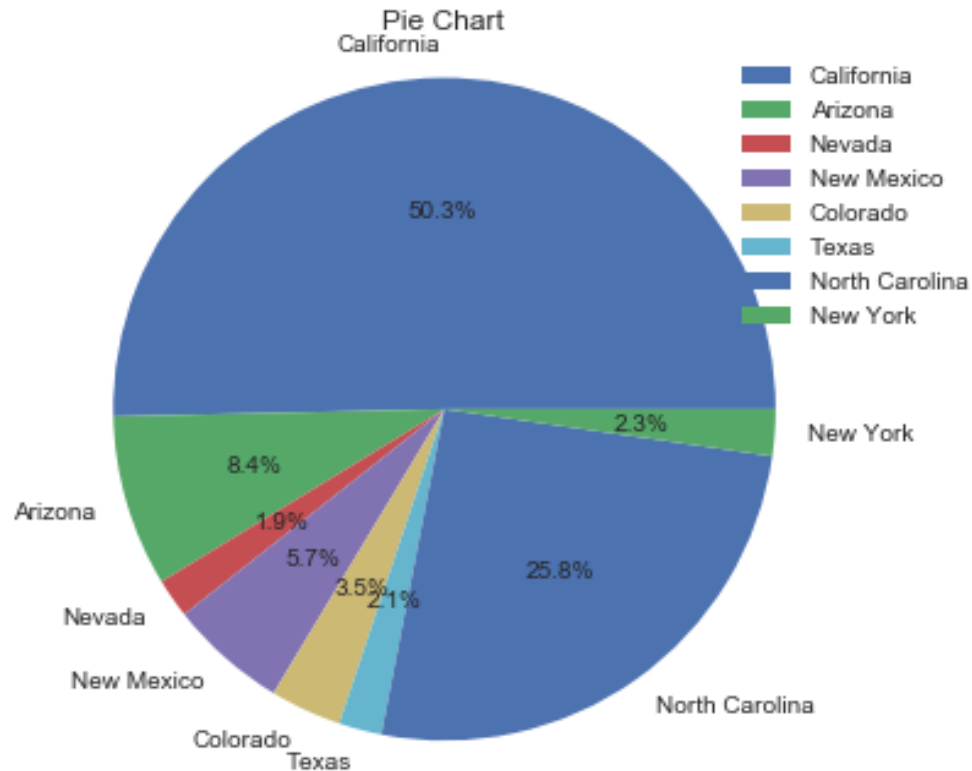
```
[18]: fig1, ax1 = plt.subplots()
ax1.pie(df['Number of Solar Plants'], labels=df.index, autopct='%1.1f%%')
ax1.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.legend()
plt.title('Pie Chart')
plt.show()
```



1.2.2 Saving the Figure

We can save the the created graph using the `savefig()` function.

```
[19]: fig1, ax1 = plt.subplots()
ax1.pie(df['Number of Solar Plants'], labels=df.index, autopct='%1.1f%%')
ax1.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.legend()
plt.title('Pie Chart')
plt.savefig('images/pie.png',dpi=fig.dpi)
```



The image is saved to `images` folder as we can see in the screenshot below



There are a lot of graph(s) that we can create using matplotlib. It is not possible to discuss everything that can be done using matplotlib in a single tutorial. Now we will focus on the graphs that can be created with matplotlib that is used often in machine learning.

1.3 Matplotlib in Machine Learning

There is a saying that **picture speaks thousands words** and it is very true in machine learning. Various kind of graph can help us in various stages of building a machine learning model. We will discuss some of the plots that can help us in each stage of building the machine learning model.

1.3.1 Understanding Data

To build any machine learning data we need to understand the data. Based on the data we can take decisions like what transformation to perform, what algorithm should we choose and what can

be some optimal hyperparameters. For Example we will be taking iris dataset.

```
[20]: import matplotlib.pyplot as plt
import pandas as pd
from sklearn import datasets
```

```
[21]: iris = datasets.load_iris()
```

```
[22]: iris_df = pd.DataFrame(iris.data,columns=iris.feature_names)
```

```
[23]: iris_df['target']=iris.target
```

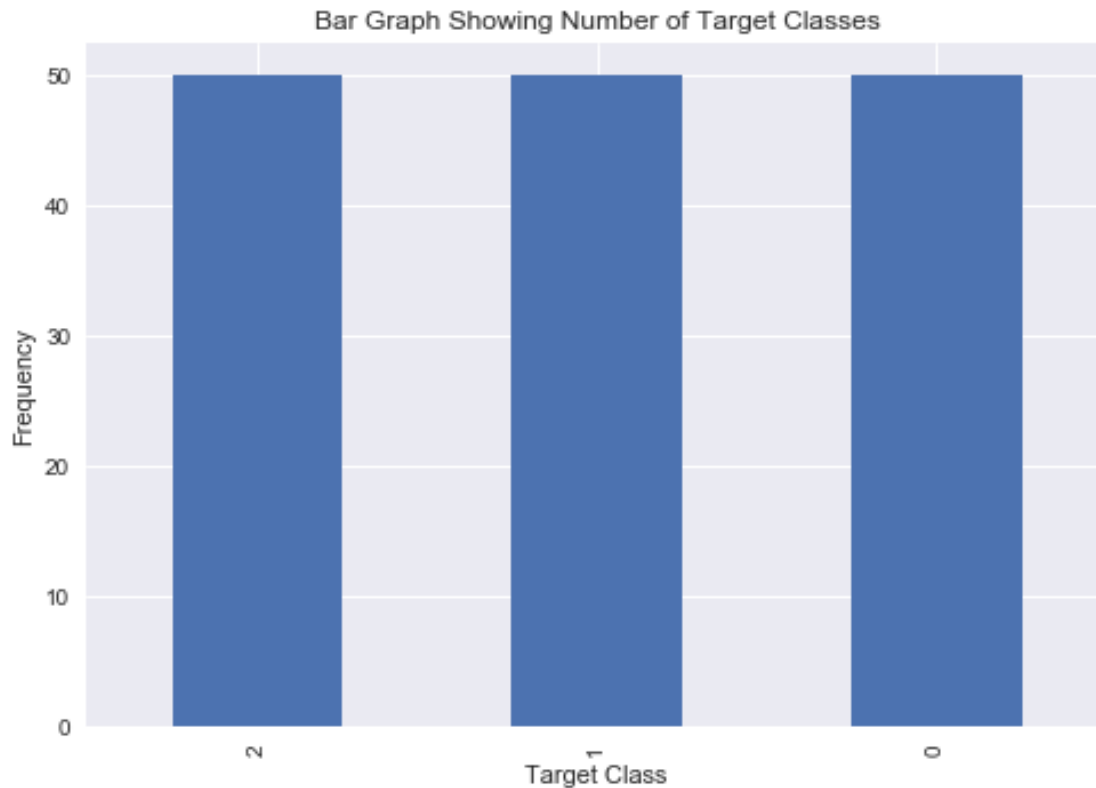
```
[24]: iris_df.head()
```

```
[24]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

```
[25]: iris_df['target'].value_counts().plot(kind='bar')
plt.title('Bar Graph Showing Number of Target Classes')
plt.xlabel('Target Class')
plt.ylabel('Frequency')
plt.show()
```



From the above bar graph we can see that there are three target class. There is no class imbalance problem in the dataset. If we had example one class very high and other very few then we would have to think about making the dataset balanced.

In cluster analysis we can look at the data and say how many cluster there could be

```
[26]: import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
plt.style.use('seaborn')
X1, Y1 = make_blobs(n_features=4, centers=3, random_state=10)
plt.scatter(X1[:, 0], X1[:, 1], marker='o', c=Y1, s=80, cmap='rainbow')
plt.title("Example Data Which Have Three Clusters")
plt.show()
```



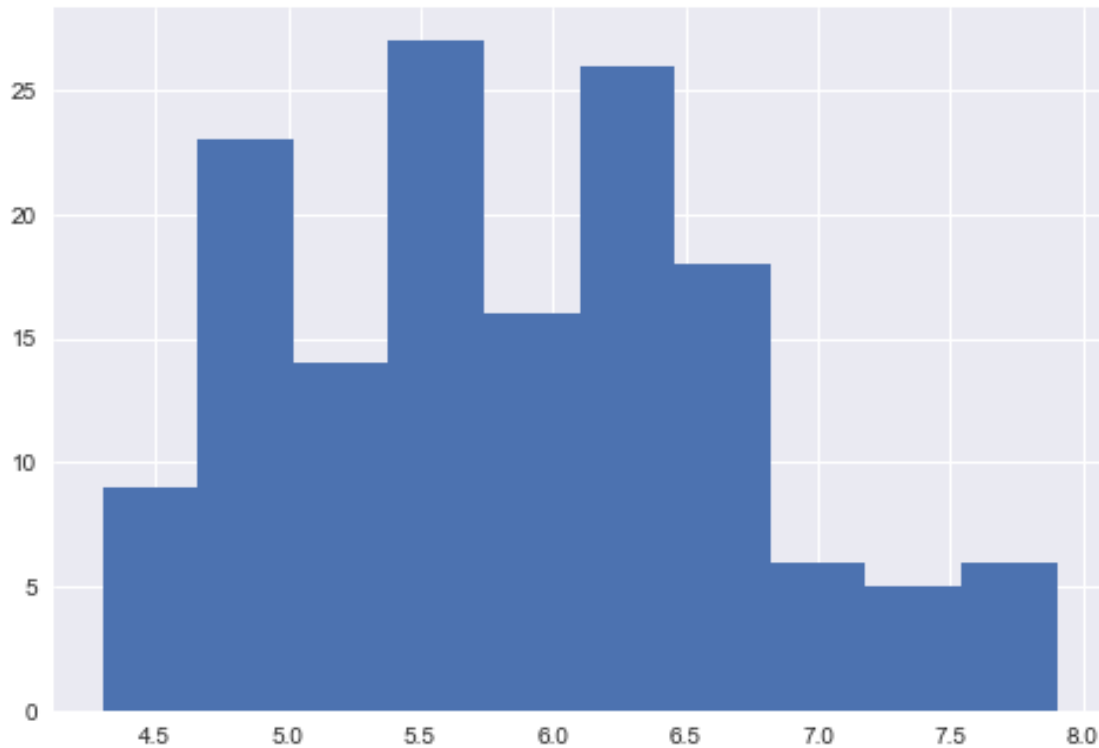
We can also plot the correlation matrix between the variables to show the correlation among the variables.

```
[40]: corr = iris_df.corr()  
corr.style.background_gradient(cmap='coolwarm')
```

```
[40]: <pandas.io.formats.style.Styler at 0x1c953671b38>
```

We can use Histogram to check the distribution of the data. Some of the machine learning algorithm have assumptions that the data should follow certain kind of distribution. For example in linear regression data should follow the normal distribution

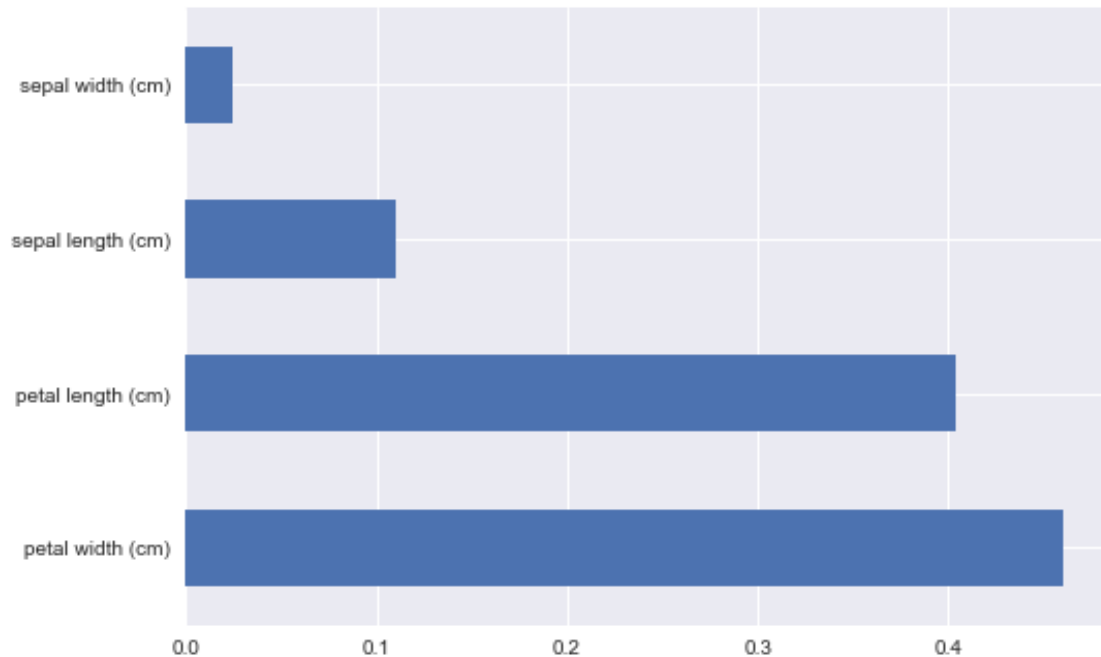
```
[28]: plt.hist(iris_df['sepal length (cm)'])  
plt.show()
```



1.3.2 Feature Selection

Feature selection is one of the important steps in machine learning. We can plot the relative importance of the features to see which are the dominant one. It will help us choose the best features from the higher number of features.

```
[29]: from sklearn.ensemble import RandomForestClassifier
      rf = RandomForestClassifier()
      rf.fit(iris.data, iris.target)
      importances = rf.feature_importances_
      feat_importances = pd.Series(importances, index=iris.feature_names)
      feat_importances.nlargest(6).plot(kind='barh')
      plt.show()
```



We can see that the `petal width(cm)` is the most important features. Most of the time the decision to keep and leave features is decided on the basis of domain knowledge.

1.4 Model Evaluation

We can plot various graphs to evaluate a machine learning. We will discuss some of the graphs below

```
[30]: from sklearn import metrics
      from sklearn.ensemble import RandomForestClassifier
      from sklearn.model_selection import train_test_split

[31]: X_train,X_test,y_train,y_test=train_test_split(iris.data,iris.
      ↪target,test_size=0.2,random_state=0)

[32]: rf=RandomForestClassifier()

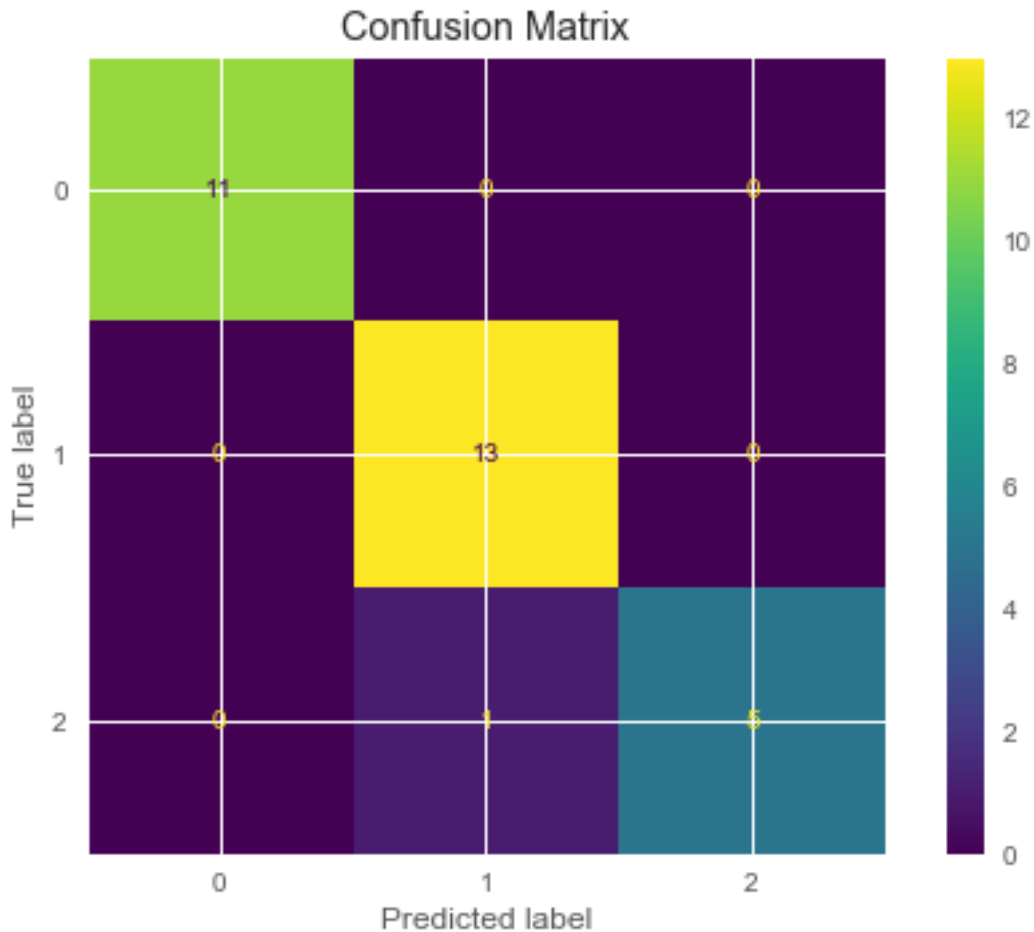
[33]: rf.fit(X_train,y_train)

[33]: RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                             criterion='gini', max_depth=None, max_features='auto',
                             max_leaf_nodes=None, max_samples=None,
                             min_impurity_decrease=0.0, min_impurity_split=None,
                             min_samples_leaf=1, min_samples_split=2,
                             min_weight_fraction_leaf=0.0, n_estimators=100,
                             n_jobs=None, oob_score=False, random_state=None,
```

```
verbose=0, warm_start=False)
```

```
[34]: y_pred=rf.predict(X_test)
```

```
[35]: plt.style.use('ggplot')
cm = metrics.confusion_matrix(y_test,y_pred)
metrics.plot_confusion_matrix(rf,X_test,y_test)
plt.title("Confusion Matrix")
plt.show()
```



1.5 Comparision of Models

We can bar graph to show the accuracy of Multiple machine learning model to see which one performed the best. The example below shows 7 classifier being trained. We will plot the bar graph showing accuracy of each model.

```
[36]: from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC, LinearSVC, NuSVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
↳ GradientBoostingClassifier
from sklearn.naive_bayes import GaussianNB

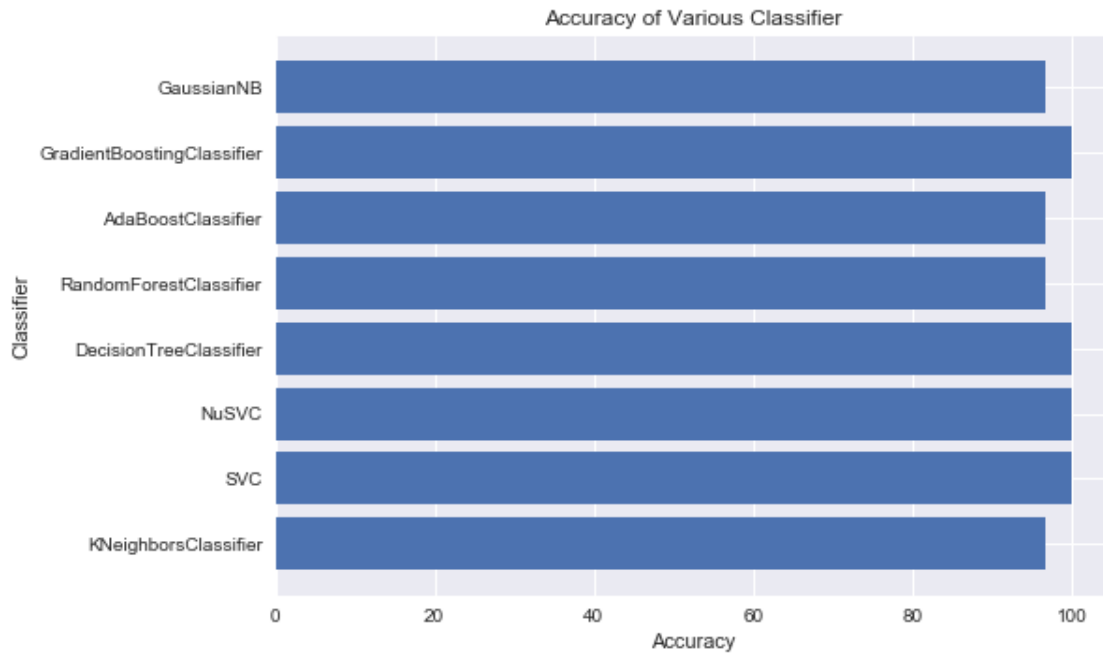
classifiers = [
    KNeighborsClassifier(3),
    SVC(kernel="rbf"),
    NuSVC(probability=True),
    DecisionTreeClassifier(),
    RandomForestClassifier(),
    AdaBoostClassifier(),
    GradientBoostingClassifier(),
    GaussianNB()]

# Logging for Visual Comparison
log_cols=["Classifier", "Accuracy"]
log = pd.DataFrame(columns=log_cols)

for clf in classifiers:
    clf.fit(X_train, y_train)
    name = clf.__class__.__name__
    y_pred = clf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    log_entry = pd.DataFrame([[name, acc*100]], columns=log_cols)
    log = log.append(log_entry)
```

```
[37]: log=log.set_index('Classifier')
```

```
[38]: plt.style.use('seaborn')
plt.barh(log.index,log['Accuracy'])
plt.xlabel('Accuracy')
plt.ylabel('Classifier')
plt.title('Accuracy of Various Classifier')
plt.show()
```



From the graph above we can see that Decision Tree Classifier, Support Vector Classifier and NuSVC model gave the best accuracy.

1.6 Conclusion

Data visulization is a important part of data analysis and machine learning. In this tutorial we learned various features of Matplotlib library. We creatd different kind of plot and explored how can those graph help us in building the best machine learning for the given dataset.